Informe de Diseño e Implementación

UdeABook: Amigos para siempre

Desafío II - Informática II

Semestre 2026 - 2

UdeABook: Amigos para siempre

Juan Camilo Restrepo Cepeda

Dorian Restrepo Vélez

Un enfoque humano, técnico y eficiente para el desarrollo de redes sociales en C++

1. Introducción y Contextualización

A través de las páginas que siguen, queremos contar el apasionante viaje que dio lugar al desarrollo de UdeABook. Frente a este desafío, entendimos que rápidamente que las redes sociales son mucho más que bonitas interfaces; mueven más de 210.000 millones de dólares cada año al conectar vidas humanas. Lo que buscamos fue llevar esa complejidad abstracta a la consola, creando el motor lógico para una empresa que quiere ser líder en Colombia y, eventualmente, a nivel mundial [2, 3].

A diferencia de proyectos anteriores, este reto evaluó nuestra capacidad real para aplicar la Programación Orientada a Objetos utilizando. Nos pusimos los sombreros de arquitectos de software para diseñar clases, controlar el encapsulamiento y construir relaciones sólidas. En este informe, vamos a detallar cómo hemos construido este ecosistema virtual, logrando la combinación de la calidez de la interacción social con la rigurosidad técnica de la gestión de memoria en el Heap.

2. Análisis del Problema y Consideraciones de Diseño

Al analizar los requerimientos iniciales el mayor desafío técnico que visualizamos fue la gestión de una cantidad indeterminada de usuarios, fotos y mensajes. Al principio pensamos que tendríamos que construir nuestras listas dinámicas desde cero con punteros dobles, tal como lo habíamos pensado en los primeros bocetos del diagrama de clases.

Sin embargo, tras una revisión de las reglas de desarrollo, se nos permitió utilizar la **Standard Template Library (STL)**, además de herramientas como **Vector** y `String`.

El equilibrio entre STL y Memoria Dinámica

El uso de vectores y de cadenas de texto estándar, hizo nuestro código mucho más legible y seguro. No bajamos la guardia, sin embargo. Vimos que si guardábamos objetos pesados (como un `Usuario` completo) directamente por valor en un `Vector`, cada vez que el vector se redimensionara duplicaría la información y saturaría la memoria.

Por lo tanto, tomamos una decisión técnica crítica: utilizaríamos la STL, pero nuestros vectores almacenarían **punteros** (ej. `vector<Usuario>`). De este modo, cada usuario, álbum o mensaje se instanciará por el operador `new` para que viva en la memoria dinámica y los vectores sólo guardarán las direcciones de memoria livianas. Esto cumple con el requisito de alta eficiencia.

3. Arquitectura del Sistema: Modelado de Entidades

Dividimos el problema en módulos independientes siguiendo nuestro diagrama de clases para darle vida a UdeABook. Las responsabilidades de cada entidad son claras y únicas:

La entidad central: El Usuario

El `Usuario` constituye la base de la plataforma. Empleamos un estricto encapsulamiento (variables `private`) para resguardar sus datos: nombre de usuario, fecha de nacimiento, correo electrónico, ciudad, país y ruta de foto de perfil. Para poder administrar su red, le pasamos varios `vector` que contienen punteros a sus amigos, mensajes enviados, recibidos, solicitudes y álbumes.

Gestión de Multimedia: Álbumes y Fotos

En UdeABook las fotos no son imágenes pesadas en RAM, abstrajimos el concepto. De cada `Foto` conocemos la fecha, el tamaño, el conteo de likes y, sobre todo, su ruta absoluta en notación Linux terminada en `.png`.

Las fotos se agrupan en una clase `Album` con una relación de composición. Un álbum cuenta con nombre, ruta de almacenamiento y una configuración de privacidad esencial para la lógica de visualización: puede ser público, privado o sólo para amigos.

4. Lógica de Interconexión: La Clase Mensaje

Lo que hace viva una red social es la comunicación. En el módulo de mensajería, el cliente indicó que los usuarios pueden mandarse textos de hasta 512 caracteres. Fue un hermoso ejercicio de optimización modelar la clase `Mensaje`.

Evitar la Redundancia de Datos

En vez de copiar los datos del emisor y del receptor del mensaje, la clase `Mensaje` simplemente contiene dos punteros: `Usuario* emisor` y `Usuario* receptor`.

Cuando Juan envía un mensaje a María, se crea **un único** objeto `Mensaje` en el Heap. Guardamos ese mismo puntero tanto en el `vector<Mensaje*>` de mensajes enviados de Juan como en el de mensajes recibidos de María. Esto, aparte de ahorrar memoria, nos daba una ventaja enorme para el control de lectura: el mensaje tiene un atributo booleano de estado (leído o no leído). Cuando María lea el mensaje y marque su estado como leído, Juan automáticamente lo verá reflejado, porque ambos apuntan a la misma dirección de memoria.

5. El Controlador Central: UdeABookGestor y Grafos

Una orquesta no puede funcionar sin un director. Hemos creado la clase `UdeABookGestor` para que se encargue de toda la red social. Este gestor dispone de un `vector<Usuario*>` con todos los registros cargados del almacenamiento permanente en el inicio del programa. Queremos hacer notar que el disco duro no se usa para procesamiento durante la ejecución, todo vive en la memoria RAM.

Matriz de adyacencia: Sugerencias de amistad

Una de las características más interesantes era proponer gente como amigos potenciales en función de amigos en común. Si fuéramos iterando por los vectores de amigos de cada usuario una y otra vez, la complejidad del algoritmo se volvería muy grande.

Siguiendo la recomendación investigativa del proyecto, realizamos la implementación de una **matriz de adyacencia**. En el gestor, definimos un arreglo bidimensional dinámico (`int** matrizAdyacencia`) en donde las filas y columnas representan a los usuarios. Un "1" en el cruce significa que son amigos. Esta estructura de teoría de grafos permite que el cálculo de las conexiones indirectas sea una operación matemática rápida y elegante, lo que nos permite ordenar el top de sugerencias casi al instante.

6. Experiencia de Usuario: Algoritmos Cotidianos

Detrás de cada clic en la consola, hay algoritmos diseñados para humanizar la experiencia del usuario.

Algoritmo de Login Ponderado

Al iniciar sesión, la plataforma debía mostrar aleatoriamente 3 fotos de perfil de amigos. Pero no basta con tener cualquier amigo, la afinidad cuenta. Se calcularon los amigos en común utilizando nuestra matriz de adyacencia y los pesos que nos piden:

- Probabilidad simple para 10 o menos amigos compartidos.
- Prioridad 2.5 veces mayor para usuarios con de 11 a 20 amigos en común.
- Prioridad 3.5 veces mayor para aquellos con 21 o más amigos en común.

Mediante librerías de aleatoriedad estándar, conseguimos que la pantalla de inicio siempre te muestre a tus contactos más cercanos.

El Carrusel Circular

Para entrar al perfil de un amigo y pasar sus álbumes, se debían mostrar las fotos en "modo carrusel" (pasando una tras otra en forma circular). En vez de mover elementos en el `vector` de fotos, usamos aritmética modular (%). En la última foto, si el usuario avanza, el índice calculado vuelve matemáticamente a 0, brindando un ciclo infinito sin consumir recursos adicionales.

7. Control, Medición de Recursos y Eficiencia

El reto era programar y demostrar la eficiencia real de nuestro código en cada funcionalidad. Para esto, incluimos un sistema de telemetría dentro de nuestro `UdeABookGestor` utilizando dos variables fundamentales.

Número de Iteraciones

Para medir la complejidad temporal se implementó una variable `contadorIteraciones`. Este valor lo vamos incrementando a través de nuestros bucles (por ejemplo, cuando buscamos mensajes en un rango de fechas). Una vez terminada la acción, mostramos el número en pantalla. El uso de `vector` nos permitió contar con iteradores eficientes, manteniendo este número lo más bajo posible.

Medición de Memoria Consumida

Se crea la variable `memoriaConsumidaBytes`. Cada vez que creamos un objeto con `new`, le añadimos su tamaño con `sizeof()`. Pero fuimos más lejos: tal como lo estipula el requisito, calculamos dinámicamente el peso de las variables locales y los parámetros por valor (como los `string` pasados a las funciones) involucrados en el scope de la funcionalidad ejecutada. Esto nos mostró claramente nuestra huella en la RAM.

8. Problemas de Desarrollo y Conclusiones

El camino para construir UdeABook estuvo lleno de aprendizajes.

Problemas Afrontados

Uno de los dolores de cabeza más grandes fue entender a fondo como la STL interactúa con punteros. Al principio, intentamos borrar elementos de nuestro `vector` utilizando los métodos normales de borrado, pero esto solo eliminaba el puntero del vector, dejando el objeto "huérfano.[en] el Heap, lo cual provocaba fugas de memoria (memory leaks). Nos vimos obligados a disciplinarnos para llamar explícitamente a`delete` antes de remover cualquier puntero de la colección. Otro desafío era pasar grandes `string` a las funciones; descubrimos el paso por referencia constante (`const string&`) para impedir copias masivas que hacían estallar nuestro contador de memoria.